

voxelcraft — codebase & documentation audit

Artifact audited: `uploads/codebase_dump.txt` (3,589,452 bytes, 60,225 lines, 169 `FILE:` entries), reconstituted into a real tree at `/home/user/repo`. **Method:** mechanical, not read-only — the tree was extracted to disk, built, linted and tested.

0. Headline

#	Finding	Severity	Confidence
1	10 tests fail in 2 targets (<code>lod_stream</code> , <code>shadow_grid</code>) — one shared root cause	High (in the artifact), Low (as an engine bug)	Certain it fails; certain of the cause
2	<code>--no-shadow-pass</code> is documented as a revert and is not one (README, EXPERIMENTS)	High — silently invalidates A/B recipes	Certain
3	"Seven shader experiments off by default" is false — <code>shadow_pass</code> defaults true and is forced true in 3 places	High	Certain
4	"Render lab" page documents 4 controls; the code renders 2. Two settings are unreachable in the UI, and one of them is a latent non-toggle	Medium (dead code) / High (doc drift)	Certain
5	<code>CLAUDE.md</code> claims clippy is "silent since batch 22" — it emits 26 warnings	Medium	Certain
6	The export tool that produced this dump silently deletes <code>src/bin/</code> — and two test guards stand down exactly where it does	High for anyone treating the dump as the repo	Certain
7	<code>.gitattributes</code> 'line-ending claims cannot be checked from this dump (the exporter normalises them)	Informational	Certain
8	Widened test bound: <code>transitions_are_symmetric_and_never_uncover</code> accepts 3.99x asymmetry	Low/known	Certain

Clean: `cargo check --all-targets` → 0 errors, 0 warnings. **47 of 49 test targets green.**
Rust↔WGSL shared-constant audit: 22/22 agree. Flag parser ↔ `--help`: 152/152 agree.

1. Test suite: 372 pass, 10 fail

```
$ cargo test --release --no-fail-fast
error: 2 targets failed:
  `--test lod_stream`
  `--test shadow_grid`
```

target	result
tests/lod_stream.rs	3 passed, 6 failed
tests/shadow_grid.rs	0 passed, 4 failed
other 47 binaries	369 passed, 0 failed
total	372 passed / 10 failed (exit 101)

All 10 failures are the *same panic*:

```
thread '...' panicked at tests/lod_stream.rs:36:5:
world never settled
thread '...' panicked at tests/shadow_grid.rs:44:5:
world never settled
```

Root cause — diagnosed, not guessed

Both files contain a **duplicated copy** of the same helper:

```
fn settle(m: &mut ChunkManager, w: &mut World, cam: Vec3) {
    for _ in 0..6000 {
        m.update(cam, w, Duration::from_millis(50));
        if m.is_settled(w) { return; }
        std::thread::sleep(Duration::from_micros(200));
    }
    panic!("world never settled");
}
```

I instrumented the identical call (`tests/zzdiag.rs`, since deleted) against the same `LodConfig::default()` and camera:

```
i=      0 t=    0.000s selected=  840 resident=    0 in_flight=  64
i=    1000 t=    0.362s selected=  840 resident=   126 in_flight=  64
i=    5999 t=    2.165s selected=  840 resident=   524 in_flight=  64
NEVER SETTLED after 6000.
```

The world *does* converge — it just needs far more than 6,000 iterations on this box:

```
$ (400,000-iteration variant)
i= 260000 t= 13.604s resident= 836 selected= 840 in_flight= 4
SETTLED at i=261336 t=13.701s resident=840 selected=840
```

So: the helper is iteration-capped, not duration-capped. 840 chunks must stream in; `max_in_flight` pins at 64 the whole time; on **2 vCPUs** that needs ****261,336 iterations / 13.7 s, and the loop gives it 6,000 iterations / ~1.2 s of sleep****. On a many-core machine it passes comfortably. The panic message ("world never settled") is misleading — it never means "the engine cannot settle", only "this VM is too small".

This is corroborated by the repo's own record — `docs/ledger.md` row `101-hw` says of `lod_stream`: **"the sandbox's «world never settled» hang is hereby closed as a **sandbox artifact**: the settle driver completes in ~14 s on hardware"**, and reports `shadow_grid` as green.

Recommendation: make the helper duration-capped (e.g. 30 s wall clock) and share it (`tests/support`) instead of copying it; or at minimum say "settle budget exhausted (N/M chunks resident)" instead of "world never settled".

*Note: this makes the test suite **machine-dependent**. `CLAUDE.md` promises `cargo test --release` as the gate; on <4 cores it is red for reasons unrelated to the code.*

2. Documentation that contradicts the code

2.1 `--no-shadow-pass` is documented as a working revert. It is not.

The parser is explicit about it:

```
// src/config.rs:974-975
"--shadow-pass"      => cfg.shadow_pass = true,
"--no-shadow-pass" => { cfg.shadow_pass = true;
    eprintln!("--no-shadow-pass retired: primary shadows now always use the dedicated
mask"); }
```

and `--help` (`src/config.rs:1133`) agrees: **"--shadow-pass is default, --no-shadow-pass is retired."** `BATCH-LOG.md:90` agrees too: **"`--no-shadow-pass` warns and does not restore primary inline tracing."**

But:

- `README.md:68` — **"Try `--isolate-glass` and/or `--shadow-pass` (revert with their `--no-`**

counterparts)."* → the `--no-` counterpart does not revert. README's own framing two paragraphs earlier (`README.md:105`) is that each flag is *"the A/B partner for a single feature — most reproduce the build from before that feature shipped, bit for bit"*; `--no-shadow-pass` is a member of that set that does not.

- `EXPERIMENTS.md:17` — the "CLI revert" column lists `--no-shadow-pass` for the dedicated shadow pass.
- `EXPERIMENTS.md:120-126` — the sample A/B recipes are built on this:

```

cargo run --release -- --bench-frames 240 --no-isolate-glass --no-shadow-pass # "control"
cargo run --release -- --bench-frames 240 --isolate-glass --no-shadow-pass # "split"
cargo run --release -- --bench-frames 240 --shadow-pass
...
cargo run --release -- --screenshot control.png --no-isolate-glass --no-shadow-pass
cargo run --release -- --screenshot split.png --isolate-glass --shadow-pass

```

Rows 1–2 still differ (in `isolate_glass`), but *every recipe in that block runs with the shadow pass on*, so EXPERIMENTS.md's own architecture table — whose `Off | Off` row is *"Combined resolve architecture"* — has an **unreachable first row**. The `control.png` / `split.png` pair on lines 125–126 is the clearest case: it is presented as combined-vs-split, and both arms have the dedicated shadow pass enabled.

2.2 "off by default" is false for `shadow_pass`

- `README.md:65` — *"The seven shader experiments are **off by default** and title-only"*
- `EXPERIMENTS.md:7` — *"...implemented, independently selectable and **off by default**"*
- `MENU-NOTES.md:13` — *"seven **off-by-default** shader switches"*
- `MENU-NOTES.md:59` — *"the two new architecture options stay off"*

The code:

site	code
<code>src/config.rs:69</code> <code>1</code>	<code>shadow_pass: true,</code> (in <code>Config::default</code>)
<code>src/settings.rs:</code> <code>58</code>	<code>... isolate_glass: c.isolate_glass, **shadow_pass: true**, glass_reflect: c.glass_reflect ...</code>
<code>src/settings.rs:</code> <code>167</code>	<code>p.shadow_pass=true;</code> — forced unconditionally at the end of <code>Preferences::decode</code>

So one of the seven is on by default, is saved as on, and is re-forced on every settings load. `src/settings.rs:167` in particular silently overrides a file that says `shadow_pass=false`.

This is **deliberate** (`BATCH-LOG.md` DS01: *"The dedicated primary shadow path is now mandatory..."*) — the defect is that README / EXPERIMENTS / MENU-NOTES were never updated to match.

2.3 The "Render lab" page lists 4 controls; the code renders 2

- `MENU-NOTES.md:26` — `| Render lab | Compact shade_hit schedule, separate glass pass, dedicated shadow pass, soft shadows |`
- `EXPERIMENTS.md:13-18` — a 4-row table whose heading is `"Start the application, then open Settings → Render lab"`.

```
// src/menu.rs:54-57
Page::Experimental => vec![
    (Setting::CompactShade, "Compact shade_hit / A-B", on(p.compact_shade_hit)),
    (Setting::SoftShadows, "Soft shadows / screen space", on(p.soft_shadows)),
],
```

`Setting::IsolateGlass` and `Setting::ShadowPass` exist in the enum (`src/menu.rs:12`), are listed in `experimental()` (`:15`), are handled in `adjust()` (`:110`, `:111`), are round-tripped by `encode/decode`, and are preserved by `keep_experiments_from` — but `"nothing ever emits an Action::Adjust for them"` (`items()` builds those rows only from `rows()`, and the keyboard path only re-uses the focused item's action). They are unreachable dead code.

Consequences worth naming:

- `src/menu.rs:111` is a latent bug: `Setting::ShadowPass=>p.shadow_pass=true`, while all three of its neighbours use `p.x!=p.x`. If the row is ever wired up, the shadow pass can be switched on and never off. It is currently harmless only because the arm is unreachable.
- **The test suite enshrines the two-row shape:**
`tests/experiments.rs:57` asserts `assert_eq!(m.rows().len(), 2)` for `Page::Experimental`, so the test and the docs disagree and the test is currently on the code's side.
- **Coverage gap:** `tests/settings_menu.rs::adjustment_controls_cannot_escape_validation_ranges` iterates `m.rows()` only, so neither hidden setting is ever range-checked.

(This is also documented as intentional in `BATCH-LOG.md` DS01: `"The obsolete glass-split and dedicated-shadow checkboxes were removed from Render lab. That page now contains Compact shade_hit and Soft shadows."` — so fix the docs, not the code. But the stale `Setting::*` plumbing is still worth deleting or wiring.)

2.4 `CLAUDE.md`'s clippy claim is false

```
$ cargo clippy --release --all-targets
# CLAUDE.md:54 says: "# silent since batch 22; keep it that way"
```

It emits **26 warnings** — 14 in the library, 12 in tests:

file	count	lint
<code>src/menu.rs</code> (110–120)	13	<code>clippy::suspicious_assignment_formatting</code> (<code>p.x!=p.x</code> , style only)
<code>src/render/mod.rs:1918</code>	1	<code>clippy::too_many_arguments</code> (12/7) — the known deliberate constructor
<code>tests/settings_menus.rs</code>	6	3 × <code>field_reassign_with_default</code> , 2 × <code>suspicious_assignment_formatting</code> , 1 × <code>... = (- ...)</code>
<code>tests/experiments.rs</code>	3	<code>field_reassign_with_default</code>
<code>tests/split_passes.rs</code>	1	<code>field_reassign_with_default</code>
<code>tests/tree.rs:324</code>	1	<code>clippy::identity_op</code> on <code>9 % 16</code>
<code>tests/emitter_gate.rs:145</code>	1	<code>clippy::assertions_on_constants</code>

The repo's own ledger disagrees with `CLAUDE.md` and with reality in *both* directions. `docs/ledger.md` row `101g-h` says clippy was *“taken to **three** pre-existing deliberate shapes (a 12-arg constructor, `9 % 16` and `PREFIX_MASK` documentation pins)”*. Those three are indeed present — but 23 more have accumulated since. `CLAUDE.md` says the command prints nothing.

3. The dump is incomplete, and the tool that made it is the reason

`gemini-code-1789442870285.py` (shipped in the tree as “the original project export utility”):

```
IGNORE_DIRS = {".git", "target", "build", "node_modules",
               ".vscode", ".idea", "bin", "obj", "__pycache__"}
IGNORE_EXTS = {".png", ".jpg", ".jpeg", ".ico", ".bin", ".exe",
               ".dll", ".so", ".dylib", ".pdb", ".lock", ".wasm"}
```

`"bin"` is meant to prune compiled Windows output. It also prunes `src/bin/` — so `src/bin/harness.rs`, `src/bin/probe.rs` and `src/bin/shaderstats.rs` are *“absent from every dump this tool produces”*, even though `docs/ledger.md` row `102-driver` records them as landed and `docs/harness.md:8` documents them.

This matters because the two guards that should catch a missing binary both stand down:

1. `tests/bins.rs` opens each file with

```
let Some(h) = source("src/bin/harness.rs") else { return; };
```

— i.e. it **passes vacuously** when the file is absent. All 4 tests report green while asserting nothing. A missing binary is indistinguishable from a correct one.

2. `tests/docs.rs::every_relative_markdown_link_resolves` explicitly skips any target containing `src/bin/` (the skip list is `.png`, `.jpg`, `research`, `src/bin/` — which is very close to the exporter's blind spots).

Net effect on this artifact: `cargo run --release --bin probe`, `--bin shaderstats` and the whole `-bin harness` fixture driver (`capture`, `validate`, `implies`, `bitexact`, `bench`, `march`, `reference`, `compare`) **cannot be run or checked**. Treat "missing `src/bin/`" as a "**dump artifact, not a repo defect**" — but the vacuous `bins.rs` guard is a real defect in the tree.

The same two constants explain every other "missing file" the docs reference: `screenshots/*.png`, `docs/look-reference-{river,bank}.png`, `voxelcraft-pre*.exe`, `harness-base.exe` and `Cargo.lock`.

3.1 `.gitattributes` cannot be verified from this dump

`.gitattributes` makes a very specific claim:

This tree is deliberately mixed: Cargo.toml, CLAUDE.md, PERF.md and README.md are CRLF, while every .wgsi is LF and .rs and docs/ are LF with exactly three exceptions — src/lib.rs, src/stats.rs and docs/batch-11-godrays.md...

This is not testable here, and I am not reporting it as an error. The exporter reads and writes in Python text mode, so it normalises line endings to the host's convention — the dump contains **60,225 `\n` and 60,225 `\r\n`**, i.e. it is 100 % CRLF, with zero mixed files. Every line-ending statement about the repo is therefore unverifiable from this artifact.

Two notes that survive the caveat anyway: the file's claim is about a third exception, `docs/batch-11-godrays.md`, that **does not exist anywhere in the tree** (no `batch-11-*` file is present), and `docs/pitfalls.md:73` says "Four of the five PDFs in there" in `docs/research/` while `docs/research-review.md:3-7` says "Seven deep-research documents were commissioned..." and that the heading "said seven until an eighth arrived". Three different counts for one directory.

4. Other defects in the tree

4.1 A test bound was widened to fit a known defect

```
// tests/lod_stream.rs:336-340
// Ledger rows 100-101 recorded pre-existing asymmetry variance (3.23x - 3.67x); bound
// at 4.0x
assert!(ratio < 4.0, "retreat pops {ratio:.2}x as much as approach ({back} vs {fw
d}))");
```

The test is named `transitions_are_symmetric_and_never_uncover`. It now passes at `**3.99× asymmetry**`, and the comment states plainly that the 4.0 was chosen to accommodate a measured defect rather than the defect fixed. Ledger rows 100/101 record the same numbers (`"transitions asymmetry read 3.23x, the ladder's variance confirmed: 5.81 -> 2.94 -> 3.23", and "3.65x" / "3.67x at pre-101"`). Either the defect should be filed in `docs/errors.md` as open, or the assertion should carry the real bound.

4.2 Dead modulus

```
// tests/tree.rs:324
assert_eq!(n.water_line, 9 % 16, "mixed cell keeps the land top");
```

`9 % 16` is identically `9`. If the intent was a 4-bit field mask it should be `9 & 15` (clippy flags this as `identity_op`). Harmless today, but the `% 16` reads as a real constraint that is not being tested. (This is one of the three "deliberate" shapes in ledger `101g-h`, so it is a known stylistic pin rather than an accident — but it is still not checking anything.)

4.3 A retained, known-broken fixture

`validation/glass-transport-results.txt` is a **failing** log kept on purpose:

```
AssertionError: (3.5, 0.0, 512.0, 0.0)
```

i.e. `primary_t=3.5, transmitted_material=0, transmitted_t=512 (= frame.far), water_leg_hit=0` where a water hit was expected. `BATCH-LOG.md` `GLO1` and `HANDOFF.md` both say glass is *"changed but unconfirmed, not declared fixed"*. `validation/native-glass-results.txt` contains only `/bin/ash: line 12: cargo: command not found`. **This is honestly documented** — listed here so it is not mistaken for something the audit found.

5. Checks that came back clean (so they need not be redone)

- `cargo check --all-targets` — **0 errors, 0 warnings**.
- **22 constants defined in both Rust and WGSL** — all values identical (`FLAG_*` × 19, `NO_CHUNK`, `UNIFORM_BIT`, `FACE_LAYER_BITS`); the only textual differences are `u` suffixes and `0xFFFF_FFFF` vs `0xFFFFFFFFu`.
- **Flag parser** ↔ `--help` — every one of the **152** flags the parser accepts appears in the help text, and no flag matches two match-arms.
- `--no-*` **revert-flag sweep** — the only arm whose body sets a field to `true` is `--no-shadow-pass` (deliberate, documented) and `--no-exit-save` (double-negative field name, correct). No other "control" is silently inert.
- **Documented "clears" semantics** (`--no-biomes` → tint/foilage/meadow) are implemented as

described, via `worldgen::has_foliage` / `has_meadow` and `scene.rs`'s `cfg.tint && cfg.biome` s.

- `tests/docs.rs` — 6/6 green: every *relative markdown link* resolves, every flag is in `--help`, no test count is transcribed, README quotes no measurement, README stays under the flag budget.
- **Key bindings** — README's control list matches `src/app.rs:619-628` (F3 stats, F4 scale, F5 heatmap, F6 Hi-Z, F7 TAA), `src/player.rs:207-226` (F fly, `Digit1` — `Digit0`, scroll hotbar) and `src/app.rs:566-601` (LMB/RMB/MMB).
- `src/settings.rs` **persistence** — bounded reads (64 KiB, `take(65537)` after a metadata check), sibling-temp-then-rename save, version gates that reject v1/v2 files carrying architecture keys — all match `MENU-NOTES.md`'s description.

6. Reproduction

```
cd /home/user/repo
export PATH=$HOME/.cargo/bin:$PATH

cargo check --all-targets                # clean
cargo clippy --release --all-targets     # 26 warnings (CLAUDE.md claims 0)
cargo test --release --no-fail-fast      # 372 passed, 10 failed, 2 targets re
d
cargo test --release --test lod_stream -- --nocapture # settle diagnostics
```

7. Suggested fixes, in priority order

1. `README.md:65-68` and `EXPERIMENTS.md:7,17,120-126` — correct the `--no-shadow-pass` "revert" and the "off by default" claims, and rewrite the sample A/B recipes: as written, every arm in them already has the dedicated shadow pass enabled.
2. `tests/lod_stream.rs` + `tests/shadow_grid.rs` — replace the duplicated `settle()` with one duration-capped helper in `tests/support/` and a diagnostic panic message.
3. `tests/bins.rs` — fail (or `#[ignore]`) when `src/bin/*.rs` is absent instead of returning early.
4. `src/menu.rs` — either render the two missing controls or delete `IsolateGlass` / `ShadowPass` from `Setting`, `experimental()` and `adjust()`; fix `p.shadow_pass=true` → `!=p.shadow_pass` if kept.
5. `MENU-NOTES.md:26` — list the two controls the page actually has (or fix the page).
6. `CLAUDE.md:54` — replace "silent since batch 22" with the real count, or clear the 23 new warnings.
7. `gemini-code-1789442870285.py` — do not prune `bin` depth-blind; match compiled-output paths only, and add `--include-bin` / a summary of skipped directories so the next dump is auditable.

8. `.gitattributes` — the third listed exception (`docs/batch-11-godrays.md`) does not exist; re-verify the list against a real checkout (it cannot be checked from a dump).